

A theoretical approach to FlashReducability

Amaru Cuba Gyllensten

March 2026

Abstract

Recent advances in memory-efficient neural network layers, such as FlashAttention, often appear as specialized engineering solutions but share a common mathematical structure. We show that many of these kernels can be understood as folds over monoids, with especially strong consequences in the commutative case. This perspective unifies MapReduce-style computation with modern deep learning optimizations. Building on this, we introduce the Local Gradient Property, which provides a sufficient condition under which gradients of monoidal folds can be computed locally from the final monoidal product and individual factors, enabling efficient backward passes. We extend this to scans over sequences, which enables efficient, parallel, backward passes that can reduce the object scanned in the backward pass, improving span and communication in nontrivial cases. We also extend it to mixed-axis folds over scans, where the scan produces significant internal state that is then folded over; regaining some of the memory efficiency of folds for scans.

1 Introduction

This work introduces sufficient conditions for efficient parallel folds and scans in deep learning.

If the operation can be formalized as a monoidal reduction or scan, it admits an efficient, parallel, forward pass (MapReduce, Blelloch Scans).

Furthermore, if the monoid satisfies the Local Gradient Property, it admits an efficient, parallel, backward pass for *reductions*.

If the LGP-derivative admits an finalize-embed-factorization, the operation admits an efficient, parallel, backward pass for *scans*.

See A.1 in the appendix for a note on the notation used throughout this paper.

2 Related work

Memory efficient kernels blabla Chen et al. (2016), Griewank and Walther (2000), blablabl flashattention Dao et al. (2022); Dao (2023). Blablabla xentropy Hsu et al. (2024); Wijmans et al. (2024). blablabl MapReduce Dean and

Ghemawat (2008), Rush et al. (2024). Most similar in spirit is perhaps Charlier et al. (2021), Schenck et al. (2022), and Bruun et al. (2023).

3 The Local Gradient Property

First we introduce the Local Gradient Property (LGP), which comes in two versions:

- The General Local Gradient Property, for non-commutative monoidal folds.
- The Commutative Local Gradient Property, for commutative monoidal folds.

Both versions show that gradient computation for monoidal folds can be simplified and made memory-efficient: if the derivative of the binary operation can be computed based on its output and individual input, then a similar local gradient rule holds for monoidal products.

Proposition 3.1 (The Local Gradient Property). *Let T be a finite-dimensional vector space and $(T, \odot)$ a monoid with a derivative $\frac{da \odot b \odot c}{db}$ that can be expressed as a function (D) of $a \odot b \odot c$, $a \odot b$ and b :*

$$D : T \times T \times T \rightarrow T^* \rightarrow T^* \\ \frac{da \odot b \odot c}{db} = D(a \odot b \odot c, a \odot b, b) \quad (1)$$

Then for any global product $g = x_1 \odot x_{1+1} \odot \cdots \odot x_N$, prefix $p = x_1 \odot x_{1+1} \odot \cdots \odot x_n$, and local slice $l = x_m \odot x_{m+1} \odot \cdots \odot x_n$, such that p is a prefix of g and l is a suffix of p (i.e. $1 \leq m \leq n \leq N$), the following holds:

$$\frac{dg}{dl} = D(g, p, l) \quad (2)$$

Proof. The result follows immediately from the monoid laws (See A.2) $\square$

Corollary 3.2 (The Commutative LGP). *In the commutative case, the prefix p can be ignored: if the derivative $\frac{da \odot b}{da}$ can be expressed as a function (D) of $a \odot b$ and a , then:*

$$\frac{dg}{dl} = D(g, l)$$

Proof. The result follows immediately from commutativity and the monoid laws (See A.3) $\square$

Corollary 3.3 (Binary form of the General LGP). *If $\frac{da \odot b}{da} = D_l(a \odot b, a)$, and $\frac{da \odot b}{db} = D_r(a \odot b, b)$, we can derive the ternary form used in proposition 3.1 using the chain rule:*

$$\begin{aligned} \frac{da \odot b \odot c}{db} &= \frac{da \odot b}{db} \circ \frac{da \odot b \odot c}{da \odot b} \\ &= D_r(a \odot b, b) \circ D_l(a \odot b \odot c, a \odot b) \\ D(g, p, l) &= D_r(p, l) \circ D_l(g, p) \end{aligned}$$

4 The Finalize–Embed Factorization

The Local Gradient Property establishes that the derivative of a monoidal product with respect to any local factor can be computed from global, prefix, and local quantities. Given a sequence $[x_1, x_2, \dots, x_N] : S^N$ of local factors, with prefix products $p_t = x_1 \odot x_{1+1} \odot \dots \odot x_t$, the LGP states that $\frac{dp_t}{dx_i} = D(p_j, p_t, x_i)$ for $t \geq i$. For folds, where we only run the backwards pass w.r.t. p_N , i.e. $\frac{d\ell}{dx_i} = D(p_N, p_i, x_i) \left[\frac{d\ell}{dp_N} \right]$, this poses no problem. However, if we apply the LGP for scans, where we need to account for $\frac{d\ell}{dp_t}$ for all ts , we quickly encounter a problem:

$$\frac{d\ell}{dx_i} = \sum_{t \geq i} D(p_t, p_i, x_i) \left[\frac{d\ell}{dp_t} \right]$$

meaning that the gradient calculation for a local factor x_i need access to all future prefixes p_t and their gradients $\frac{d\ell}{dp_t}$. To address this limitation, we factor the local gradient function D_S through an auxiliary gradient accumulation (partial) semigroup $(A, \oplus)$ ¹. using functions finalize and embed:

$$\begin{aligned} \text{finalize}(p, l) &: A \rightarrow S^* \quad (\text{For general monoids}) \\ \text{finalize}(l) &: A \rightarrow S^* \quad (\text{For commutative monoids}) \\ \text{embed}(g) &: S^* \rightarrow A \end{aligned}$$

such that the local gradient satisfies:

$$\begin{aligned} D_S(g, p, l) &= \text{finalize}(p, l) \circ \text{embed}(g) \\ \sum_{t=i}^J D_S(p_t, p_i, x_i) \left[\frac{d\ell}{dp_t} \right] &= \text{finalize}(p_i, x_i) [a_i] \\ \text{where } a_i &= \bigoplus_{t=i}^J \text{embed}(p_t) \left[\frac{d\ell}{dp_t} \right] \end{aligned}$$

¹The accumulator A need not be a vector space. We require only a partially defined associative operation $\oplus$ sufficient for the suffix accumulations that arise in the backward pass. In additive and GL_N -transported cases, $\oplus$ is ordinary addition. For max-like scans, A may instead carry suffix attribution data, and $\oplus$ is defined only for compatible attribution objects.

in the general case (and corresponding prefix-less formulation in the commutative case), resulting in a local gradient of the form

$$\frac{d\ell}{dx_i} = \text{finalize}(p_i, x_i)[a_i]$$

from which it follows that local gradient calculation can be performed efficiently using suffix scans over embedded A s.

5 Representation, Transport, Factorization, and Execution

While we won't provide an exhaustive characterization of when operations satisfy the LGP and when they have a finalize-embed-factorization, we do identify one broad and important class for the general case, one for the commutative case, and a general way to construct new LGP-satisfying operations from existing ones:

- If T satisfies the LGP, then any S that admits a faithful smooth homomorphism $S \hookrightarrow T$ with an inverse on its image satisfies the LGP. Furthermore, if T has a finalize-embed factorization through A , then so does S .
- Any monoid admitting a faithful smooth finite dimensional homomorphism into GL_N with an inverse on its image satisfies the General LGP and has a finalize-embed factorization.
- Any monoid admitting a faithful smooth finite dimensional homomorphism into $(\mathbb{R}^N, +)$ with an inverse on its image satisfies the Commutative LGP and has a finalize-embed factorization.

We begin by proving the former, from which the latter two follow. We also discuss a nonsmooth LGP-style rule for max under a chosen tie-counting subgradient convention.

Proposition 5.1 (Transport of the General LGP). *Let $(S, \odot)$ and $(T, \otimes)$ be monoids, and let*

$$\phi : S \rightarrow T$$

be a faithful smooth homomorphism $(S, \odot) \hookrightarrow (T, \otimes)$ with ϕ a smooth inverse on its image:

$$\psi : \phi(S) \rightarrow S \quad (\psi \circ \phi = \text{id})$$

If $(T, \otimes)$ satisfies the General Local Gradient Property with derivative rule D_T , then so does $(S, \odot)$ with

$$D_S(g, p, l) := \frac{d\phi(l)}{dl} \circ D_T(\phi(g), \phi(p), \phi(l)) \circ \frac{d\psi(\phi(g))}{d\phi(g)} \quad (3)$$

Proof. The result follows directly from the homomorphism properties and the chain rule (See appendix A.4). $\square$

Corollary 5.2 (Factorization transport). *With the same conditions as in proposition 5.1, if D_T can be factored into embed_T and finalize_T , then so can D_S , with:*

$$\begin{aligned}\text{embed}_S(g) &= \text{embed}_T(\phi(g)) \circ \frac{d\psi(\phi(g))}{d\phi(g)} \\ \text{finalize}_S(p, l) &= \frac{d\phi(l)}{dl} \circ \text{finalize}_T(\phi(p), \phi(l))\end{aligned}$$

Proof. The result follows directly by substituting the factorization $D_T = \text{finalize}_T \circ \text{embed}_T$ in Proposition 5.1 and grouping the composition by associativity. $\square$

Corollary 5.3 (Commutative case). *In the commutative case of proposition 5.1, transport takes the form:*

$$\begin{aligned}D_S(g, l) &= \frac{d\phi(l)}{dl} \circ D_T(\phi(g), \phi(l)) \circ \frac{d\psi(\phi(g))}{d\phi(g)} \\ \text{embed}_S(g) &= \text{embed}_T(\phi(g)) \circ \frac{d\psi(\phi(g))}{d\phi(g)} \\ \text{finalize}_S(l) &= \frac{d\phi(l)}{dl} \circ \text{finalize}_T(\phi(l))\end{aligned}$$

Proof. The result follows directly from the homomorphism properties and the chain rule (See appendix A.5). $\square$

Extension to smooth manifolds. For simplicity, the General Local Gradient Property, the Commutative Local Gradient Property, and the two transport propositions were stated for finite-dimensional vector spaces. However, the same arguments extend directly to finite-dimensional smooth manifolds equipped with smooth monoid operations, provided all derivatives are interpreted as maps between tangent spaces and their pullbacks on cotangent spaces. The proofs are unchanged, since they use only associativity, smoothness, and the chain rule.

Note on the identity element While the LGP is stated for monoids, many common operations in deep learning (such as the binary operations in Attention or certain RNNs) are naturally semigroups where an identity element is not immediately present. However, any semigroup $(S, \odot)$ can be extended to a monoid by adjoining an identity element 1 such that $a \odot 1 = 1 \odot a = a$. In neural network layers, such an identity usually corresponds to a "null" initial state or a masked value whose derivatives need not be computed. In such cases, we can adjoin a purely formal identity element 1 and allow $D(g, l)$ (or $D(g, p, l)$) to remain undefined at $l = 1$, since these expressions are only used for $l \neq 1$.

5.1 $GL_N(\mathbb{R})$ satisfies the General LGP

Given $a, b \in GL_N(\mathbb{R})$ with $a \odot b := ab$,

$$\begin{aligned} D_l(a \odot b, a)[\Delta] &= \Delta (a^{-1} (a \odot b))^{\top} \\ D_r(a \odot b, b)[\Delta] &= ((a \odot b) b^{-1})^{\top} \Delta \end{aligned}$$

which gives that

$$D(g, p, l)[\Delta] = (pl^{-1})^{\top} \Delta (p^{-1}g)^{\top} \quad (4)$$

with factorization through the additive matrix semigroup $(M_N(\mathbb{R}), +)$:

$$\begin{aligned} \text{finalize}_{GL_N(\mathbb{R})}(p, l)[e] &= l^{-\top} p^{\top} e p^{-\top} \\ \text{embed}_{GL_N(\mathbb{R})}(g)[\Delta] &= \Delta g^{\top} \end{aligned}$$

By the transport proposition, any monoid $(S, \odot) \hookrightarrow GL_N(\mathbb{R})$ admits a transported General LGP formulation using the following concrete instances of finalize and embed:²

$$\text{finalize}_S(p, l)[e] = \frac{d\phi(l)}{dl} [\phi(l)^{-\top} \phi(p)^{\top} e \phi(p)^{-\top}] \quad (5)$$

$$\text{embed}_S(g)[\Delta] = \frac{d\psi(\phi(g))}{d\phi(g)} [\Delta] \phi(g)^{\top} \quad (6)$$

5.2 $(\mathbb{R}^N, +)$ satisfies the Commutative LGP

A simpler and quite common special case of $GL_N(\mathbb{R})$ is $(\mathbb{R}^N, +)$. We show that this satisfies the Commutative LGP.

$$\begin{aligned} a \odot b &:= a + b \\ D_{\mathbb{R}^N}(a \odot b, a) &= \frac{da + b}{da} \\ &= \text{id} \end{aligned}$$

with factorization through the additive vector semigroup $(\mathbb{R}^N, +)$:

$$\begin{aligned} \text{finalize}_{(\mathbb{R}^N, +)}(l) &= \text{id} \\ \text{embed}_{(\mathbb{R}^N, +)}(g) &= \text{id} \end{aligned}$$

²Often, the image $\phi(S)$ is considerably restricted, as in the case of affine transforms with scalar scaling. While it embeds into $GL_{D+1}(\mathbb{R})$, it can be represented by $D+1$ parameters, rather than the $(D+1)^2$ of the full $GL_{D+1}(\mathbb{R})$. As such, the transport should be seen as a proof of existence, with computation being performed in a more performant and/or numerically stable representation. This extends to the gradient accumulator object A .

By transport, any monoid $(S, \odot) \hookrightarrow (\mathbb{R}^N, +)$ satisfies the commutative LGP with the following concrete instances of finalize and embed:

$$\text{finalize}_S(l) = \frac{d\phi(l)}{dl} \quad (7)$$

$$\text{embed}_S(g) = \frac{d\psi(\phi(g))}{d\phi(g)} \quad (8)$$

5.3 $(\mathbb{R}, \max)$ satisfies a Commutative LGP-style rule

A slightly more involved case is max, using tie-counting subgradient convention, i.e. $S := \{m : \mathbb{R}, t : \mathbb{N}_{>0}\}$ with $S^* \cong \mathbb{R}$, since cotangents are taken only with respect to the real-valued max output m ; the tie count t is discrete auxiliary state and receives no cotangent³. The tie-counting max operation is defined as

$$a \odot b = \begin{cases} \{m : a_m, t : a_t + b_t\} & \text{if } a_m = b_m \\ a & \text{if } a_m > b_m \\ b & \text{if } b_m > a_m \end{cases}$$

where

$$D(g, l)[\Delta] = \begin{cases} \frac{l_t}{g_t} \Delta & \text{if } g_m = l_m \\ 0 & \text{if } g_m > l_m \end{cases} \quad (9)$$

Factoring through the following partial semigroup⁴ $(A, \oplus)$, embed, and finalize:

$$\begin{aligned} A &:= \{m : \mathbb{R}, \gamma : \mathbb{R}\} \\ a \oplus b &= \begin{cases} \{m : a_m, \gamma : a_\gamma + b_\gamma\} & \text{if } a_m = b_m \\ a & \text{if } a_m < b_m \end{cases} \\ \text{embed}(g)[\Delta] &= \left\{ m : g_m, \gamma : \frac{1}{g_t} \Delta \right\} \\ \text{finalize}(l)[e] &= \begin{cases} l_t e_\gamma & \text{if } l_m = e_m \\ 0 & \text{if } l_m < e_m \end{cases} \end{aligned}$$

This generalizes to $\mathbb{R}^N$ with pointwise max and corresponding pointwise A .

³Strictly speaking, this uses a chosen subgradient convention rather than the smooth proposition as stated.

⁴Note that the operation $\oplus$ is defined only for compatible ordered pairs with $a_m \leq b_m$; while not total, it is total for suffix scans over embedded prefix-max states.

5.4 Execution carrier

In practice, performing the fold using the semantic monoid $(S, \odot_S)$ can lead to numerical instability or performance issues. Instead, we often perform the fold in an execution carrier $(M, \odot_M)$, which acts as a numerically stable or hardware-efficient representation of S .

We define an execution carrier $(M, \odot_M)$ for a semantic monoid $(S, \odot_S)$ as a monoid homomorphism $\Psi : (M, \odot_M) \rightarrow (S, \odot_S)$ with a right inverse $\Phi : S \rightarrow M$ such that

$$\begin{aligned}\Psi \circ \Phi &= \text{id}_S \\ \Psi(a \odot_M b) &= \Psi(a) \odot_S \Psi(b) \quad \forall a, b \in M\end{aligned}$$

From which it follows that $\odot_i a_i = \Psi(\odot_i \Phi(a_i))$, then, if S is an LGP-satisfying monoid, we can compute the fold or scan in M , and utilize the LGP-backwards pass for S .⁵

6 Efficient Folds

For efficient monoidal folds we present a tiled, parallel, sketch of the forward pass using a fused `map_fold` : $X_{\text{tile}} \rightarrow S_{\text{summary}}$ function which computes local monoidal summaries S_{summary} from local input tiles X_{tile} , corresponding to the following non-tiled function:

$$\text{map_fold}(x)_i = \bigodot_j l_{i,j} \quad (10)$$

where $l_{i,j} = \text{map}(x_{i,j})$ is a monoidal factor corresponding to one input interaction. In the following pseudocode, bold symbols like **g**, **p**, **l**, and **x** denote tiled quantities, with $\mathbf{x}_{b,r}$ being the tile at index (b, r) , while the non-bold symbols above denote non-tiled quantities, with $x_{i,j}$ denoting the interaction at index (i, j) . The parallel, tiled, forward pass takes the following form:

```
def FoldForwardPass( $x \in X$ )  $\rightarrow S_{\text{summary}}^B$  :
  lb,r :  $S_{\text{summary}}$   $\leftarrow$  map_fold(tile( $x$ )b,r)
  if Commutative: // Parallel reduction.
    gb :  $S_{\text{summary}}$   $\leftarrow \odot_r$  lb,r
  if General: // Parallel exclusive prefix scan.
    pb,r :  $S_{\text{summary}}$   $\leftarrow \odot_{t < r}$  lb,t
    optionally store pb,r for backward pass
    gb  $\leftarrow$  pb,R_{\text{last}}  $\odot$  lb,R_{\text{last}}
  return gb
```

⁵While superficially similar to the transport homomorphism, the execution homomorphism is subtly different: $M \rightarrow S$ is a *surjective* homomorphism, whereas $S \hookrightarrow T$ is an *injective* homomorphism.

For the backward pass, we use fused backward functions $\text{fused_bwd} : X_{\text{tile}} [\times S_{\text{summary}}] \rightarrow A_{\text{summary}} \rightarrow X_{\text{tile}}^*$ that take an input tile (and exclusive prefix summary tile in the general case), a gradient accumulation summary A_{summary} , and produces local gradients X_{tile}^* , corresponding to the following non-tiled function:

$$\text{fused_bwd}(x, p^{\leftarrow})[e] = \sum_{i,j} \frac{d l_{i,j}}{d x} [\text{finalize}(p_{i,j}, l_{i,j})[e_i]] \quad (11)$$

$$\text{fused_bwd}(x)[e] = \sum_{i,j} \frac{d l_{i,j}}{d x} [\text{finalize}(l_{i,j})[e_i]] \quad (12)$$

where $p_{i,j}$ corresponds to the tile-local view of a global running prefix, computed using the left-boundary $p^{\leftarrow}$ and l :

$$p_{i,j} = p_i^{\leftarrow} \odot \left(\bigodot_{t \leq j} l_{i,t} \right) \quad (13)$$

The parallel, tiled, backward pass takes the following form

```
def FoldBackwardPass( $x \in X$ ,  $\mathbf{g} \in S_{\text{summary}}^B$ ,  $\Delta \in S_{\text{summary}}^{B*}$ )  $\rightarrow X^*$ :
   $\mathbf{x}_{b,r} : X_{\text{tile}} \leftarrow \text{tile}(x)_{b,r}$ 
   $\mathbf{e}_b : E_{\text{summary}} \leftarrow \text{embed}(\mathbf{g}_b)[\Delta_b]$ 
  if Commutative:
     $\delta_{b,r} : X_{\text{tile}}^* \leftarrow \text{fused\_bwd}(\mathbf{x}_{b,r})[\mathbf{e}_b]$ 
  if General:
    load or recompute  $\mathbf{p}_{b,r}$ 
     $\delta_{b,r} : X_{\text{tile}}^* \leftarrow \text{fused\_bwd}(\mathbf{x}_{b,r}, \mathbf{p}_{b,r})[\mathbf{e}_b]$ 
  // Parallel sum-reduce (tiling adjoint).
  return  $\text{tile}^*[\delta_{b,r}]$ 
```

7 Efficient Scans

For efficient monoidal scans we present a tiled, parallel, sketch of the forward pass using a fused function $\text{map_scan} : X_{\text{tile}} \rightarrow S_{\text{tile}}$ which is the scan analogue of map_fold : it computes a tile-local monoidal scan over a full intermediate tile S_{tile} from local input tiles X_{tile} , corresponding to the following non-tiled function:

$$\text{map_scan}(x)_{i,j} = \bigodot_{t \leq j} l_{i,t} \quad (14)$$

where $l_{i,j} = \text{map}(x_{i,j})$ is a monoidal factor corresponding to one input interaction.

```

def ScanForwardPass( $x \in X$ )  $\rightarrow S_{\text{tile}}^{B \times R}$ :
   $\mathbf{h}_{b,r} : S_{\text{tile}} \leftarrow \text{map\_scan}(\text{tile}(x)_{b,r})$ 
  // Parallel exclusive prefix scan
   $\mathbf{p}_{b,r} : S_{\text{summary}} \leftarrow \bigodot_{t < r} \mathbf{h}_{b,t_{\text{last}}}$ 
  store  $\mathbf{p}_{b,r}$  for backward pass
  return  $\mathbf{p}_{b,r} \odot \mathbf{h}_{b,r}$ 

```

For the backward pass, we use fused backward functions $\text{fused_bwd} : X_{\text{tile}} [\times S_{\text{tile}}] \rightarrow E_{\text{tile}} \rightarrow X_{\text{tile}}^*$ that take an input tile (and prefix scan tile in the general case), a global suffix scanned gradient accumulation tile E_{tile} , and produce local gradients X_{tile}^* , corresponding to the following non-tiled function:

$$\text{fused_bwd}(x, y)[e] = \sum_{i,j} \frac{dl_{i,j}}{dx} [\text{finalize}(y_{i,j}, l_{i,j})[e_{i,j}]] \quad (15)$$

$$\text{fused_bwd}(x)[e] = \sum_{i,j} \frac{dl_{i,j}}{dx} [\text{finalize}(l_{i,j})[e_{i,j}]] \quad (16)$$

as well as a function $\text{embed_scan} : S_{\text{tile}} \rightarrow S_{\text{tile}}^* \rightarrow E_{\text{tile}}$, that take a global prefix scanned tile S_{tile} , a global gradient tile S_{tile}^* and produce tile-local suffix scanned gradient accumulation tile E_{tile} :

$$\text{embed_scan}(y)[\Delta]_{i,j} = \bigoplus_{t \geq j} \text{embed}(y_{i,t})[\Delta_{i,t}] \quad (17)$$

Backward scan:

```

def ScanBackwardPass( $x \in X$ ,  $\mathbf{y} \in S_{\text{tile}}^{B \times R}$ ,  $\Delta \in S_{\text{tile}}^{B \times R^*}$ )  $\rightarrow X^*$ :
   $\mathbf{x}_{b,r} : X_{\text{tile}} \leftarrow \text{tile}(x)_{b,r}$ 
   $\mathbf{e}_{b,r} : E_{\text{tile}} \leftarrow \text{embed\_scan}(\mathbf{y}_{b,r})[\Delta_{b,r}]$ 
   $\mathbf{e}_{b,r}^{\rightarrow} : E_{\text{summary}} \leftarrow \bigoplus_{t > r} \mathbf{e}_{b,t_{\text{first}}}$  // Parallel suffix scan
   $\mathbf{e}_{b,r}^{\Sigma} : E_{\text{tile}} \leftarrow \mathbf{e}_{b,r} \oplus \mathbf{e}_{b,r}^{\rightarrow}$ 
  if Commutative:
     $\delta_{b,r} : X_{\text{tile}}^* \leftarrow \text{fused\_bwd}(\mathbf{x}_{b,r})[\mathbf{e}_{b,r}^{\Sigma}]$ 
  if General:
     $\delta_{b,r} : X_{\text{tile}}^* \leftarrow \text{fused\_bwd}(\mathbf{x}_{b,r}, \mathbf{y}_{b,r})[\mathbf{e}_{b,r}^{\Sigma}]$ 
  // Parallel sum-reduce (tiling adjoint).
  return  $\text{tile}^*[\delta_{b,r}]$ 

```

7.1 Comparison with Blelloch-Style Scan Backpropagation

The scan constructions above should be compared to the usual reverse-mode treatment of a Blelloch scan. In a standard parallel backward pass, one differentiates the scan circuit itself, which amounts to scanning backward over transfer operators describing how cotangents propagate from later prefixes to earlier inputs. The LGP view instead rewrites the backward pass into three stages: a

local embedding of each upstream cotangent, a suffix scan in an auxiliary gradient accumulator, and a local finalization step. The asymptotic consequence is therefore not primarily a change in total work, but a simplification of the object scanned in the reverse pass.

For the literal additive scan on $(\mathbb{R}^N, +)$, there is no asymptotic gain: the backward pass is already just a suffix sum of N -vectors, so both the usual Blelloch backpropagation and the LGP formulation require $O(JN)$ work and $O(N \log J)$ span. In this case, the two viewpoints are essentially the same algorithm.

The $GL_N(\mathbb{R})$ case is different. A generic Blelloch-style backward scan propagates affine linear operators on $N \times N$ cotangents, so the scan combines objects using matrix multiplication, giving $O(JN^3)$ work and $O(N^3 \log J)$ span in the dense case. By contrast, the LGP factorization rewrites the backward pass as a suffix scan over ordinary $N \times N$ matrices under addition, with the matrix multiplications moved into the local embed/finalize steps. The total work remains $O(JN^3)$, but the span becomes $O(N^3 + N^2 \log J)$. Thus, for GL_N , the gain is not in total work but in reducing the cost of the scan tree itself.

More generally, the transport propositions show that the same distinction persists for monoids that are not themselves additive or matrix-valued, but admit faithful smooth homomorphisms into these representation spaces. If S transports into T with gradient accumulator A , the reverse scan can be performed over A , rather than a $S \times S$ dimensional Jacobian.

The same distinction appears in communication complexity. A parallel suffix scan over P processors requires $\Theta(\log P)$ communication rounds regardless of whether it is derived from Blelloch backpropagation or from the LGP. The difference is in the communicated state: communication volume is proportional to the size of the scan element. Thus, transport into $(\mathbb{R}^M, +)$ communicates $\Theta(M \log P)$ words per processor, while transport into $GL_M(\mathbb{R})$ communicates $\Theta(M^2 \log P)$ words per processor. These match specialized Blelloch-style backward scans for literal addition or matrix-valued states, but can be asymptotically smaller than a generic reverse scan in local coordinates on S .

Essentially, this framework gives a principled way to derive specialized backward scans for Blelloch-style parallel prefix computations by identifying a simpler algebra in which gradient accumulation can be scanned.

8 Efficient Scan-Fold

While scans do not see much memory savings, we can combine folds and scans, such that the intermediate state produced by the scan is reduced for each timestep by a commutative fold, keeping only the final folded value. Essentially fusing a Scan over R that produces $S^{B \times R}$ states with a commutative fold over B , producing final output F^R . This enables scans over wide states that are reduced before being pushed downstream, generalizing current Mamba style layers.

For this, we use a fused, tile-local, function that maps input interactions $x_{i,j}$

and scan state $p_{i,j}$ to local fold-factors $l_{i,j}$, which are folded over b :

$$\text{map_fold_inner} : X_{\text{tile}} \times S_{\text{tile}} \rightarrow F_{\text{summary}}$$

$$\text{map_fold_inner}(x, p)_j = \bigodot_i^F l_{i,j}$$

where $l_{i,j} : F = \text{map}_{X \times S \rightarrow F}(x_{i,j}, p_{i,j})$ is a local fold factor constructed from an input interaction $x_{i,j}$ and corresponding scan state $p_{i,j}$.

```

def ScanFoldForwardPass( $x \in X$ )  $\rightarrow F_{\text{summary}}^R$  :
   $\mathbf{x}_{b,r} : X_{\text{tile}} \leftarrow \text{tile}(x)_{b,r}$ 
   $\mathbf{h}_{b,r} : S_{\text{tile}} \leftarrow \text{map\_scan}(\mathbf{x}_{b,r})$ 
  // Parallel exclusive prefix scan
   $\mathbf{p}_{b,r} : S_{\text{summary}} \leftarrow \bigodot_{t < r}^S \mathbf{h}_{b,t_{\text{last}}}$ 
  optionally store  $\mathbf{p}_{b,r}$  for backward pass
   $\mathbf{p}_{b,r}^{\text{tile}} : S_{\text{tile}} \leftarrow \mathbf{p}_{b,r} \odot_S \mathbf{h}_{b,r}$ 
  // Fused local fold over (inner) B
   $\mathbf{l}_{b,r} : F_{\text{summary}} \leftarrow \text{map\_fold\_inner}(\mathbf{x}_{b,r}, \mathbf{p}_{b,r}^{\text{tile}})$ 
  // Parallel reduction over (global) B
   $\mathbf{y}_r : F_{\text{summary}} \leftarrow \bigodot_b^F \mathbf{l}_{b,r}$ 
return  $\mathbf{y}_r$ 

```

For the backward pass, we use the LGP for both the associative scan monoid S and the commutative fold F . For the commutative fold we define a fused backward mapping, analogous to fused_bwd used for efficient folds, that computes the gradient w.r.t. input values x and scan state p :

$$\text{fused_inner_bwd}(x, p)[e] = \sum_{i,j} \frac{d l_{i,j}}{d(x, p)} [\text{finalize}_F(l_{i,j})[e_j]]$$

With such a function, the scan-fold backward pass can be defined as follows:

```

def ScanFoldBackwardPass(
    x ∈ X,
    y ∈ FRsummary,
    Δ ∈ FRsummary*
) → X*:

    erF : AF,summary ← embedF(yr)[Δr]
    load pb,r
    xb,r : Xtile ← tile(x)b,r

    // 1. Reconstruct states (local)
    pb,rtile : Stile ← pb,r ⊙S map_scan(xb,r)

    // 2. Differentiate the Fold (local)
    (δxb,rfold : Xtile*, Δpb,rtile : Stile*) ← fused_inner_bwd(xb,r, pb,rtile)[erF]

    // 3. Differentiate the Scan
    // 3.1 Embed the cotangents (local)
    eb,r : Atile ← embed_scan(pb,rtile)[Δpb,rtile]

    // 3.2 Parallel (exclusive) suffix scan (global)
    eb,r→ : Asummary ← ⊕t>r eb,tfirst
    eb,rΣ : Atile ← eb,r ⊕ eb,r→

    // 4. Finalize (local)
    if Commutative:
        δxb,rscan ← fused_bwd(xb,r)[eb,rΣ]
    if General:
        δxb,rscan ← fused_bwd(xb,r, pb,rtile)[eb,rΣ]
    δxb,r ← δxb,rscan + δxb,rfold
    // 5. Total Gradient (global)
    // Parallel global sum-reduce (tiling adjoint)
    return tile*[δxb,r]

```

9 Instances

9.1 LogWeightedSum

A common monoid, used in attention and various information theoretic measures, is weighted averages with weights given in logspace: $LWS := \{z : \mathbb{R}, \mu : \mathbb{R}^D\}$, with

$$\begin{aligned}
 a \odot_{LWS} b &= \{z : \text{logaddexp}(a_z, b_z), \mu : a_\mu \cdot \exp(a_z - z) + b_\mu \cdot \exp(b_z - z)\} \\
 \bigodot_i \{z : l_i, \mu : v_i\} &= \left\{ z : \ln \left(\sum_i \exp(l_i) \right), \mu : \sum_i v_i \cdot \exp(l_i - z) \right\}
 \end{aligned}$$

That it satisfies the commutative LGP, can be seen by transport to additive $LWS_T := \{w : \mathbb{R}, s : \mathbb{R}^D\}$:

$$\begin{aligned}\phi(a) &= \{w : \exp(a_z), s : \exp(a_z) \cdot a_\mu\} \\ \psi(x) &= \left\{z : \ln(x_w), \mu : \frac{1}{x_w} \cdot x_s\right\}\end{aligned}$$

With induced gradient factorization:

$$\begin{aligned}\text{embed}_{LWS}(g)[\Delta] &:= \{z : g_z, w : (\Delta_z - \Delta_\mu \cdot g_\mu), s : \Delta_\mu\} \\ \text{finalize}_{LWS}(l)[e] &:= \{z : \exp(l_z - e_z)(e_w + e_s \cdot l_\mu), \mu : \exp(l_z - e_z)e_s\}\end{aligned}$$

The execution carrier used in FlashAttention, essentially fusing the max-trick in logaddexp over the reduction, can be defined follows:

$$\begin{aligned}LWS_M &:= \{m : \mathbb{R}, e : \mathbb{R}, v : \mathbb{R}^D\} \\ \Phi(a) &= \{m : a_z, e : 1, v : a_\mu\} \\ \Psi(x) &= \left\{z : x_m + \ln x_e, \mu : \frac{x_v}{x_e}\right\} \\ x \odot_{LWS_M} y &= \{m : h_m, e : h_e + l_e \exp(l_m - h_m), v : h_v + l_v \exp(l_m - h_m)\} \\ \text{where } (h, l) &= \begin{cases} (x, y) & x_m > y_m \\ (y, x) & x_m \leq y_m \end{cases}\end{aligned}$$

which uses fewer transcendental functions, and is numerically more stable than the original monoid when using floating point arithmetic.

9.2 Attention

For single-headed attention, we use the LWS -monoid:

$$\text{attention}(Q, K, V)_i = \left(\bigodot_j \{z : \langle Q_i, K_j \rangle, \mu : V_j\} \right)_\mu \quad (18)$$

9.3 Single-distribution information theoretic measures

For a distribution p , parameterized as unnormalized logits, we can reuse the same LWS -monoid, but with scalar μ :

$$\text{stat}(p) = \bigodot_i \{z : p_i, \mu : p_i\} \quad (19)$$

For $g = \text{stat}(p)$,

$$\begin{aligned}
g_z &= \ln \sum_i \exp(p_i) \\
g_\mu &= \sum_i \exp(p_i - g_z) \cdot p_i \\
&= \sum_i \bar{p}_i \cdot p_i \\
H(p) &= - \sum_i \bar{p}_i \ln \bar{p}_i \\
&= g_z - g_\mu
\end{aligned}$$

where $\bar{p}_i = \exp(p_i - g_z)$ denotes the global probability corresponding to logit i .

9.4 Directed two-distribution information theoretic measures

For two distributions p and q , parameterized as unnormalized logits, we use what is essentially the cross-product of LWS -monoids, defined by transport from $\{z^p : \mathbb{R}, z^q : \mathbb{R}, \mu : \mathbb{R}, \sigma : \mathbb{R}\}$ to $\{w^p : \mathbb{R}, w^q : \mathbb{R}, s : \mathbb{R}, \Sigma : \mathbb{R}\}$, with

$$\begin{aligned}
\phi(a) &= \{w^p : \exp(a_{z^p}), w^q : \exp(a_{z^q}), s : \exp(a_{z^p}) \cdot a_\mu, \Sigma : \exp(a_{z^p}) \cdot a_\sigma\} \\
\psi(b) &= \left\{ z^p : \ln(b_{w^p}), z^q : \ln(b_{w^q}), \mu : \frac{1}{b_{w^p}} \cdot b_s, \sigma : \frac{1}{b_{w^p}} \cdot b_\Sigma \right\}
\end{aligned}$$

and induced $\odot$, then:

$$\text{stat}(p, q) = \bigodot_i \{z^p : p_i, z^q : q_i, \mu : p_i, \sigma : q_i\} \quad (20)$$

From which we can compute the following, given $g = \text{stat}(p, q)$

$$\begin{aligned}
\text{KL}(p||q) &= \sum_i \bar{p}_i \cdot (\ln \bar{p}_i - \ln \bar{q}_i) \\
&= \sum_i \bar{p}_i \cdot (p_i - g_{z^p} - q_i + g_{z^q}) \\
&= g_\mu - g_{z^p} - g_\sigma + g_{z^q} \\
\text{CE}(p||q) &= H(p) + \text{KL}(p||q) \\
&= g_{z^q} - g_\sigma
\end{aligned}$$

Which can be extended to a six-element carrier for bidirectionality, enabling computation of $\text{CE}(q||p)$, $\text{KL}(q||p)$, and $\text{Jeffreys}(p, q)$ as well, or shrunk by removing the term μ when only computing $\text{CE}(p||q)$. Like the former example, we can use the same fused max-trick as in attention (albeit applied to both z^p and z^q).

9.5 One-hot Cross-Entropy

For a distribution p , parameterized as unnormalized logits, and q a one-hot distribution $q_i = \delta_{it}$, we have

$$\text{CE}(q\|p) = -\ln \bar{p}_t = \ln \left(\sum_i \exp(p_i) \right) - p_t$$

As such, the carrier needs only the log-partition of p and the logit p_t :

$$S = \{z : \mathbb{R}, \ell : \mathbb{R}\} \quad (21)$$

$$T = \{w : \mathbb{R}, l : \mathbb{R}\} \quad (22)$$

$$\phi(a) = \{w : \exp(a_z), l : a_\ell\} \quad (23)$$

$$\psi(b) = \{z : \ln(b_w), \ell : b_l\} \quad (24)$$

Then with:

$$\text{stat}(p, t) = \bigodot_i \{z : p_i, \ell : \delta_{it} p_i\} \quad (25)$$

we have that $\text{CE}(t\|p) = g_z - g_\ell$ for $g = \text{stat}(p, t)$. Like the former example, we can use the same fused max-trick as in attention.

9.6 Affine transforms

An affine transform $f(x) = f_a \cdot x + f_b$ is a noncommutative monoid under composition and, as commonly used in sequence models, reverse composition: $g(f(x)) = (g \circ f)(x) = g_a(f_a \cdot x + f_b) + g_b$, where $f \odot g = g \circ f$. This gives the carrier $\{a : \mathbb{R}, b : \mathbb{R}^D\}$, where $(f \odot g) = \{a : g_a f_a, b : g_a \cdot f_b + g_b\}$, which transports to $GL_{D+1}(\mathbb{R})$

With the following embed and finalize:

$$\begin{aligned} \text{embed}(g)[\Delta] &= \{\alpha : g_a \Delta_a + g_b^\top \Delta_b, \beta : \Delta_b\} \\ \text{finalize}(p, l)[e] &= \left\{ a : \frac{e_\alpha - p_b^\top e_\beta}{l_a}, b : \frac{p_a}{l_a} e_\beta \right\} \end{aligned}$$

References

- Bruun, L. M., Larsen, U. S., Hinnerskov, N. H., and Oancea, C. E. (2023). Reverse-mode ad of multi-reduce and scan in futhark. In *Proceedings of the 35th Symposium on Implementation and Application of Functional Languages*, pages 1–14.
- Charlier, B., Feydy, J., Glaunès, J. A., Collin, F.-D., and Durif, G. (2021). Kernel operations on the gpu, with autodiff, without memory overflows. *Journal of Machine Learning Research*, 22(74):1–6.

- Chen, T., Xu, B., Zhang, C., and Guestrin, C. (2016). Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*.
- Dao, T. (2023). Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Re, C. (2022). Flashattention: Fast and memory-efficient exact attention with IO-awareness. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K., editors, *Advances in Neural Information Processing Systems*.
- Dean, J. and Ghemawat, S. (2008). Mapreduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113.
- Elliott, C. M. (2009). Beautiful differentiation. *ACM Sigplan Notices*, 44(9):191–202.
- Griewank, A. and Walther, A. (2000). Algorithm 799: revolve: an implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Trans. Math. Softw.*, 26(1):19–45.
- Hsu, P.-L., Dai, Y., Kothapalli, V., Song, Q., Tang, S., Zhu, S., Shimizu, S., Sahni, S., Ning, H., and Chen, Y. (2024). Liger kernel: Efficient triton kernels for llm training. *arXiv preprint arXiv:2410.10989*.
- Rush, K., Charles, Z., Garrett, Z., Augenstein, S., and Mitchell, N. (2024). Drjx: Scalable and differentiable mapreduce primitives in jax. *arXiv preprint arXiv:2403.07128*.
- Schenck, R., Rønning, O., Henriksen, T., and Oancea, C. E. (2022). Ad for an array language with nested parallelism. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE.
- Wijmans, E., Huval, B., Hertzberg, A., Koltun, V., and Krähenbühl, P. (2024). Cut your losses in large-vocabulary language models. *arXiv preprint arXiv:2411.09009*.

A Appendix

A.1 Notation

Types & Terms $a : A$ denotes that the term a has type A . For example, $a : \mathbb{R}$ denotes that the term a is a real number.

Product Types $A \times B$ denotes the product type of A and B . For brevity, terms of product types are occasionally written out as tuples. If $a : A$ and $b : B$, we can write $(a, b) : A \times B$, and vice versa. For a term

$x : A \times B$, x_0 is the term corresponding to the left-hand type (A), and x_1 corresponds to the right-hand type. We also used named product types, where $\{a : A, b : B\}$ is the type of pairs of A and B indexed by the names a and b , i.e. $X : \{a : A, b : B\}$ says that X is a named product type, with two named parts a and b of type A and B respectively. Similarly, terms of named product types are written out as $x = \{a : v, b : w\}$, meaning $x_a = v$, $x_b = w$. For brevity, in expressions such as $x \mapsto \{a : x_a, b : -a\}$, we allow for internal references.

Functions $A \rightarrow B$ denotes the type of functions from A to B . Function arrows associate to the right:

$$A \rightarrow B \rightarrow C \equiv A \rightarrow (B \rightarrow C) \cong A \times B \rightarrow C$$

Function Application To clearly distinguish primal and dual operations, standard parentheses $f(x)$ denote function application where the argument x represents a parameter, primal value, or internal state. Conversely, square brackets $f[v]$ are reserved for functions acting on dual objects, such as gradients, cotangents, or gradient accumulation elements. A mapping parameterized by primal state x that acts on dual object v is written as $f(x)[v]$.

Derivatives The mathematical convention used regarding derivatives is that $\frac{dA}{dB}$ denotes the linear map that transforms gradients of A into gradients of B (backward / pullback Jacobian), i.e. if $a : A$, $b : B$, $\frac{da}{db} : A^* \rightarrow B^*$, inspired by Conal Elliott's work on Automatic Differentiation (Elliott, 2009).

Function Composition & Multiplication $\circ$ denotes function composition, so the chain rule reads $\frac{dc}{da} = \frac{db}{da} \circ \frac{dc}{db}$, corresponding to $(f \circ g)(x) = f(g(x))$. Matrix multiplication is denoted by standard juxtaposition (e.g. AB is the product of matrices A and B).

A.2 Proof for the general LGP (proposition 3.1)

Proof. Since $(T, \odot)$ is a monoid, l is a suffix of p , and p a prefix of g , there exists $a, b \in T$ such that $p = a \odot l$ and $g = a \odot l \odot b$.

Plugging this into eq. 1, we get $\frac{dg}{dl} = \frac{da \odot l \odot b}{dl} = D(a \odot l \odot b, a \odot l, l) = D(g, p, l)$. $\square$

A.3 Proof for the commutative LGP (corollary 3.2)

Proof. For any partition of H into H^1, H^2 , with corresponding partial products $p_1 := \odot_i H_i^1$ and $p_2 := \odot_i H_i^2$ and global product $g := \odot_i H_i$

$$\begin{aligned}
g &= p_1 \odot p_2 = p_2 \odot p_1 && \text{(By commutativity and associativity)} \\
\frac{dg}{dp_1} &= \frac{dp_1 \odot p_2}{dp_1} \\
&= D(p_1 \odot p_2, p_1) && \text{(By the condition } \frac{da \odot b}{da} = D(a \odot b, a)) \\
&= D(g, p_1)
\end{aligned}$$

This extends trivially to $p_1 := h_i$, i.e., $H^1 = [h_i]$:

$$\begin{aligned}
\frac{dg}{dh_i} &= \frac{dp_1}{dh_i} \circ D(g, p_1) \\
&= D(g, p_1) && \left(\frac{dp_1}{dh_i} = \text{id} \right) \\
&= D(g, h_i)
\end{aligned}$$

□

A.4 Proof for general transport (proposition 5.1)

Proof. Due to ϕ being a homomorphism with inverse ψ on its image:

$$\begin{aligned}
\phi(s_1 \odot s_2) &= \phi(s_1) \otimes \phi(s_2) \\
s_1 \odot s_2 &= \psi(\phi(s_1) \otimes \phi(s_2))
\end{aligned}$$

Let

$$\begin{aligned}
g_S &= s_1 \odot s_{1+1} \odot \cdots \odot s_N \\
p_S &= s_1 \odot s_{1+1} \odot \cdots \odot s_n \\
l_S &= s_m \odot s_{m+1} \odot \cdots \odot s_n
\end{aligned}$$

for $1 \leq m \leq n \leq N$. Let $t_i = \phi(s_i)$, and

$$\begin{aligned}
g_T &= \phi(g_S) = t_1 \otimes t_{1+1} \otimes \cdots \otimes t_N \\
p_T &= \phi(p_S) = t_1 \otimes t_{1+1} \otimes \cdots \otimes t_n \\
l_T &= \phi(l_S) = t_m \otimes t_{m+1} \otimes \cdots \otimes t_n
\end{aligned}$$

The chain rule, in combination with the homomorphism properties and the fact that $(T, \otimes)$ satisfies the LGP, gives that

$$\begin{aligned}
\frac{d g_S}{d l_S} &= \frac{d l_T}{d l_S} \circ \frac{d g_T}{d l_T} \circ \frac{d g_S}{d g_T} \\
&= \frac{d l_T}{d l_S} \circ D_T(g_T, p_T, l_T) \circ \frac{d g_S}{d g_T} \\
&= \frac{d \phi(l_S)}{d l_S} \circ D_T(\phi(g_S), \phi(p_S), \phi(l_S)) \circ \frac{d \psi(\phi(g_S))}{d \phi(g_S)} \\
&= D_S(g_S, p_S, l_S)
\end{aligned}$$

□

A.5 Proof for commutative transport (corollary 5.3)

Proof. Let $g_S = s_1 \odot s_{1+1} \odot \cdots \odot s_N$, and let l_S be the product over an arbitrary index set $I \subseteq \{1, \dots, N\}$: $l_S := \bigodot_{i \in I} s_i$. Since ϕ is a homomorphism, $g_T = \phi(g_S)$, $l_T = \phi(l_S)$.

The chain rule, in combination with the homomorphism properties and the fact that $(T, \otimes)$ satisfies the commutative LGP, gives that

$$\begin{aligned}
\frac{d g_S}{d l_S} &= \frac{d l_T}{d l_S} \circ \frac{d g_T}{d l_T} \circ \frac{d g_S}{d g_T} \\
&= \frac{d l_T}{d l_S} \circ D_T(g_T, l_T) \circ \frac{d g_S}{d g_T} \\
&= \frac{d \phi(l_S)}{d l_S} \circ D_T(\phi(g_S), \phi(l_S)) \circ \frac{d \psi(\phi(g_S))}{d \phi(g_S)} \\
&= D_S(g_S, l_S)
\end{aligned}$$

□